

KREACIJSKI PATERNI

1. SINGLETON

Ideja: Singleton patern omogućava postojanje samo jedne instance određene klase u sistemu, uz globalni pristup toj instanci. Najčešće se koristi za klase koje predstavljaju zajedničke konfiguracije ili resurse koje koristi više različitih komponenti sistema.

Primjena na naš dijagram:

- **KonfiguracijaSistema:** klasa koja sadrži zajedničke konfiguracijske podatke sistema (SMTP server za slanje email obavještenja, API ključ za obradu online plaćanja, osnovne sistemske postavke aplikacije)
- **Plaćanje i Notifikacija:** klase koje koriste *KonfiguracijaSistema* za pristup konfiguracijskim podacima

Pošto svi dijelovi sistema moraju koristiti iste konfiguracijske vrijednosti, potrebno je spriječiti kreiranje više različitih instanci klase *KonfiguracijaSistema*.

Prednosti:

- Centralizovano upravljanje konfiguracijom sistema
- Konzistentni podaci u svim dijelovima aplikacije
- Jednostavnije održavanje sistema
- Smanjeno korištenje resursa

Mane:

- Otežano testiranje zbog globalnog pristupa objektu
- Jača povezanost između klasa sistema
- Potencijalni problemi kod paralelnog rada niti (thread safety)

2. FACTORY METHOD

Ideja: Factory Method obrazac omogućava kreiranje objekata bez direktnog povezivanja ostatka sistema sa konkretnim klasama koje se instanciraju. Umjesto direktnog kreiranja objekata pomoću operatora new, proces kreiranja preuzima posebna fabrička klasa (Factory). Na taj način sistem postaje fleksibilniji, lakši za održavanje i jednostavniji za proširenje novim tipovima objekata.

Primjena na naš dijagram - Plaćanje:

- **PlaćanjeFactory:** fabrička klasa koja na osnovu odabrane metode plaćanja kreira odgovarajući objekat
- **KarticaPlaćanje, GotovinaPlaćanje, BankovniTransferPlaćanje, RatePlaćanje:** konkretne klase koje nasljeđuju baznu klasu *Plaćanje*.

Factory metoda *kreirajPlaćanje(tip : MetodaPlaćanja) : Plaćanje* prima tip plaćanja i vraća odgovarajući objekat klase plaćanja.

Primjena na naš dijagram – Notifikacije:

- **NotifikacijaFactory:** fabrička klasa koja na osnovu tipa notifikacije kreira odgovarajući objekat
- **PotvrdaRezervacijeNotifikacija, PotvrdaPlaćanjaNotifikacija:** konkretne klase koje nasljeđuju baznu klasu Notifikacija

Factory metoda *kreirajNotifikaciju(tip : TipNotifikacije)* : *Notifikacija* prima tip notifikacije i vraća odgovarajući objekat notifikacije. Na taj način ostatak sistema ne mora direktno kreirati pojedinačne tipove obavještenja, već koristi centralizovani mehanizam za kreiranje notifikacija.

Prednosti:

- Jednostavnije dodavanje novih metoda plaćanja i tipova notifikacija
- Manja zavisnost između klasa
- Lakše održavanje sistema
- Jasnije odvajanje logike kreiranja objekata
- Centralizovano kreiranje objekata
- Manje dupliranja koda

Mane:

- Veći broj klasa u sistemu
- Dodatna složenost implementacije
- Dodatni nivo apstrakcije
- Nepotrebna kompleksnost kod manjih sistema

3. ABSTRACT FACTORY

Ideja: Abstract Factory patern omogućava kreiranje porodica međusobno povezanih objekata bez direktnog specificiranja njihovih konkretnih klasa.

Primjena na naš dijagram: U trenutnoj verziji sistema ne postoji dovoljno opravdana potreba za implementacijom ovog patern. Sistem trenutno ne sadrži više kompleksnih porodica povezanih objekata, različite podsisteme koji moraju raditi zajedno niti više međusobno zavisnih grupa proizvoda. Zbog toga bi implementacija ovog patern dodatno zakomplikovala postojeći dizajn aplikacije.

Prednosti:

- Lakša zamjena kompletnih porodica objekata
- Konzistentnost između povezanih objekata
- Jednostavnije proširenje sistema

Mane:

- Veća kompleksnost sistema
- Veći broj interfejsa i klasa
- Nepotrebno komplikovanje manjih aplikacija

4. BUILDER

Ideja: Builder patern omogućava postepeno kreiranje kompleksnih objekata kroz jasno definisane korake konstrukcije.

Primjena na naš dijagram: Iako sistem sadrži klase poput *Rezervacija* i *PlanPutovanja*, njihovo kreiranje trenutno nije dovoljno složeno da bi zahtijevalo Builder patern. Objekti se kreiraju direktno putem konstruktora i ne zahtijevaju višefaznu konstrukciju, veliki broj opcionalnih koraka niti kompleksnu konfiguraciju objekta. Zbog toga Builder patern nije uveden u trenutnoj verziji sistema.

Prednosti:

- Jasno odvajanje procesa konstrukcije objekta
- Lakše kreiranje kompleksnih konfiguracija
- Poboljšana organizacija koda

Mane:

- Veći broj klasa
- Dodatna kompleksnost sistema
- Nepotrebno komplikovanje jednostavnih objekata

5. PROTOTYPE

Ideja: Prototype patern omogućava kreiranje novih objekata kopiranjem postojećih objekata (prototipa), umjesto njihovim kreiranjem od početka pomoću konstruktora. Novi objekat nastaje kloniranjem postojećeg objekta.

Primjena na naš dijagram:

- **PlanPutovanja:** klasa koja može implementirati Prototype patern za kopiranje postojećih planova putovanja
- **StavkaPlana:** prateća klasa koja se također kopira prilikom kloniranja plana

Turističke agencije često koriste veoma slične planove putovanja za iste ili slične destinacije. Umjesto da agent svaki put ručno kreira kompletan plan putovanja i sve stavke plana, moguće je napraviti kopiju već postojećeg plana i zatim izvršiti manje izmjene (kopiranje postojećeg plana, promjena datuma polaska, dodavanje ili uklanjanje pojedinih aktivnosti, prilagođavanje rasporeda). Na taj način novi plan nastaje mnogo brže i uz manje mogućnosti za greške.

Prednosti:

- Brže kreiranje sličnih planova putovanja
- Manje ručnog unosa podataka
- Lakše prilagođavanje postojećih aranžmana
- Smanjenje mogućnosti greške pri unosu
- Veća efikasnost turističkih agenata

Mane:

- Složenija implementacija kod dubokog kopiranja objekata

- Potrebno pravilno kopirati povezane objekte
- Mogući problemi ako se kopirani objekti međusobno dijele
- Dodatna logika za održavanje kloniranja

ZAKLJUČAK

- **Singleton:** centralizovano upravljanje konfiguracijom sistema (*KonfiguracijaSistema*)
- **Factory Method:** kreiranje različitih tipova plaćanja (*PlaćanjeFactory*) i notifikacija (*NotifikacijaFactory*)
- **Prototype:** kopiranje postojećih planova putovanja i kreiranje sličnih aranžmana (*PlanPutovanja*)
- **Abstract Factory:** analiziran kao moguće buduće proširenje sistema
- **Builder:** analiziran za potencijalno kreiranje kompleksnijih objekata

U našem sistemu primijenili smo **Singleton** kako bi svi dijelovi aplikacije koristili jedinstvenu konfiguraciju sistema. Također smo primijenili **Factory Method** za fleksibilno kreiranje različitih metoda plaćanja i tipova notifikacija bez direktne zavisnosti od konkretnih klasa.

Prototype patern analiziran je kao pogodno rješenje za kopiranje postojećih planova putovanja i brzo kreiranje novih turističkih aranžmana uz manje izmjene.

Paterni **Abstract Factory** i **Builder** trenutno nisu implementirani jer bi dodatno zakomplikovali sistem bez značajne koristi u postojećoj verziji aplikacije.